

TACO : COMPILATION ET GÉNÉRATION DE CODE D'EXPRESSIONS TENSORIELLES

L'algèbre tensorielle est un puissant outil de modélisation avec des applications dans le *machine learning*, l'analyse de données, l'ingénierie et la simulation numérique. Les tenseurs sont souvent creux (avec de nombreuses composantes nulles) et les expressions de calcul complexes doivent être implémentées dans un noyau unique pour des raisons de performance et de gestion mémoire. Les développeurs doivent alors optimiser manuellement des noyaux pour chacune des expressions d'intérêt avec des tenseurs associant des dimensions denses et creuses. L'explosion combinatoire du nombre d'expressions potentielles rend difficiles l'implémentation et l'optimisation de chacune. L'approche générative proposée dans le compilateur *taco*, acronyme de *tensor algebra compiler* (voir <http://tensor-compiler.org>) permet de générer automatiquement du code parallèle compétitif avec les meilleurs noyaux optimisés manuellement et surtout pour un nombre infini d'expressions tensorielles.

Les expressions d'algèbre linéaire sont au cœur des performances des logiciels de simulation du CEA-DAM. Les tenseurs sont une généralisation des vecteurs et matrices à tout nombre de dimensions, ils permettent d'effectuer des calculs multilinéaires **1**. De nombreuses applications concernent des tenseurs creux, par exemple le Amazon Reviews Tensor (3 dimensions : utilisateurs $\times$ produits $\times$ avis) peut potentiellement contenir 107 exaoctets (10^{18} octets) composantes, dont seulement 13 gigaoctets (10^9 octets) sont non nuls. La **figure 1** illustre, sur une expression simple, la complexité d'itération sur des indices creux pour optimiser le nombre d'opérations nécessaires à l'obtention du résultat.

Le nombre infini d'expressions tensorielles simples ou composées ainsi que les différences entre les architectures matérielles cibles sont les raisons du choix d'une approche par compilation retenue dans *taco*, acronyme de *tensor algebra compiler* **2**. Son implémentation nécessite les trois techniques suivantes :

- la définition du stockage des tenseurs se fait en spécifiant pour chacune de ses dimensions si elle est pleine (*D-dense*) ou creuse (*S-sparse*) et en choisissant l'ordre de ses dimensions. Ainsi le format CSR (*Compressed sparse row*) usuel pour les matrices creuses est assimilable à DS (*Dense-Sparse*). Un tenseur de dimension 4 tel que DSDD permet de stocker des matrices par blocs ;

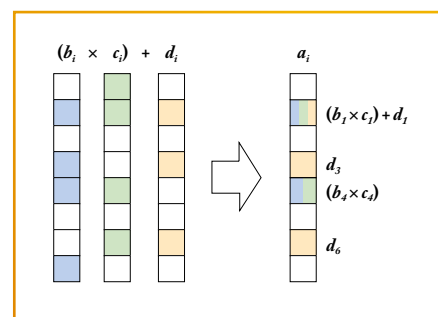


Figure 1

➤ Calcul creux de $a_i = b_i * c_i + d_i$. Les couleurs dans le vecteur résultat montrent quelles opérations et valeurs des opérandes sont combinées pour chaque valeur résultat non nulle.

- les graphes d'itérations sont la représentation intermédiaire du compilateur décrivant les espaces d'itération sur les valeurs non nulles d'un tenseur. Ce concept générique permet de représenter aussi bien des expressions simples telles qu'un SpMV (*Sparse matrix-vector multiplication*) que des expressions composées complexes comme le MTTKRP (*Matricized tensor times Khatri-Rao product*) ;

- les treillis de fusion relient les opérations avec les espaces d'itération. Si deux tenseurs sont multipliés (resp. additionnés) alors la fusion est une conjonction ($\wedge$) (resp. disjonction $\vee$), car il faut que les deux opérandes (resp. une seule) soient non nulles pour que le résultat le soit aussi (**figure 2**).

À partir d'une expression écrite en notation tensorielle **1**, le compilateur *taco* s'ap-

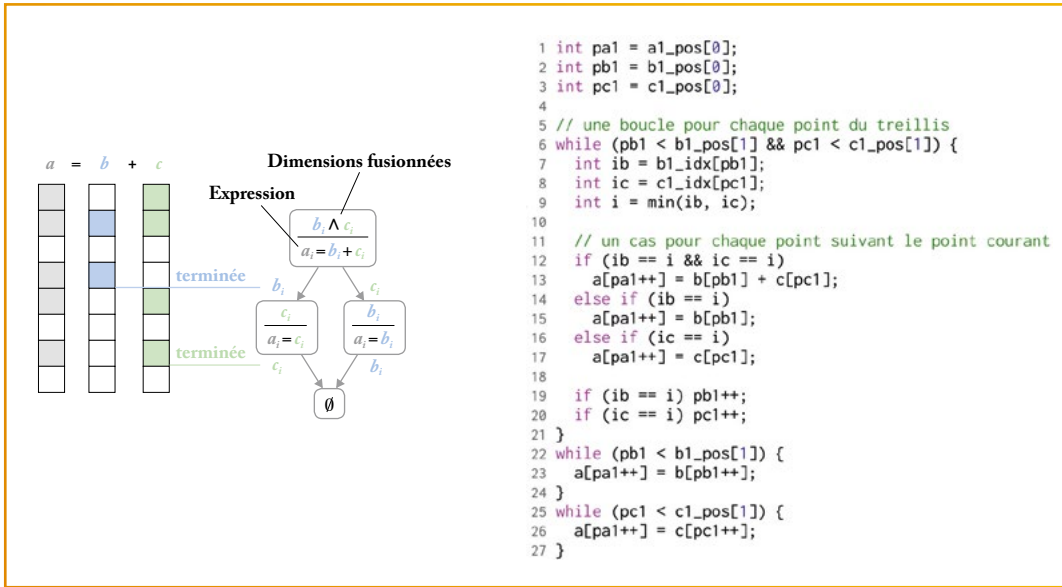


Figure 2

➤ Exemple d'une addition de vecteurs creux ($a_i = b_i + c_i$). La partie centrale montre le treillis de fusion de cette expression. La partie droite montre le code généré automatiquement par *taco* pour cette expression.

puie sur ces trois concepts pour générer du code optimisé parallèle. L'algorithme *two-way merge* 3 est utilisé pour optimiser les parcours des indices des dimensions creuses fusionnées (figure 2).

L'évaluation des performances du code généré démontre l'efficacité de cette approche tant sur des expressions simples comme un SpMV de forme $b_i = A_{ij} \times x_j$ (figure 3a) que sur des expressions complexes impliquant des tenseurs d'ordre élevé comme un MTTKRP de forme $A_{ij} = B_{ikl} \times C_{klj} \times D_{lj}$. Elle permet surtout d'éliminer le compromis entre performance et exhaustivité des bibliothèques existantes. La figure 3b montre pour l'expression $A_{ij} = B_{ij} + C_{ij} + D_{ij}$ les gains

de performance obtenus avec *taco* contre les bibliothèques qui implémentent cette expression. Le parallélisme pour des architectures à mémoire partagée est assuré par l'ajout de directives d'interfaces de programmation parallèle (telles que OpenMP ou Cilk) au niveau des boucles externes. Les travaux de recherche en cours portent sur la prise en compte de nouveaux formats de stockage et d'un niveau de parallélisme distribué.

La suite logicielle *taco* est composée de trois couches logicielles successives et utilisables indépendamment : une bibliothèque C++, un outil en ligne de commande et une interface web. L'outil en ligne de commande permet de comparer les perfor-

mances des noyaux de code générés, d'étudier l'impact du choix des formats et d'optimiser interactivement le code généré 4.

Le compilateur *taco* propose une approche générative dédiée à l'algèbre tensorielle et pour une infinité de formats de stockage. En s'appuyant sur ce compilateur, on peut construire des langages, des bibliothèques et des logiciels de simulation manipulant des tenseurs denses et creux sans écrire manuellement le code ni sacrifier les performances.

RÉFÉRENCES

- 1 G. RICCI-CURBASTRO, T. LEVI-CIVITA, « Méthodes de calcul différentiel absolu et leurs applications », *Math. Ann.*, **54**, p. 125-201 (1901).
- 2 F. KJOLSTAD, S. KAMIL, S. CHOU, D. LUGATO, S. AMARASINGHE, "The Tensor Algebra Compiler", *Proc. ACM Program. Lang.*, Vancouver, Canada, Oct. 22-27, 2017, **1**, Issue OOPSLA, Article 77 (2017).
- 3 D. KNUTH, *The art of computer programming: sorting and searching*, Pearson Education (1973).
- 4 F. KJOLSTAD, S. CHOU, D. LUGATO, S. KAMIL, S. AMARASINGHE, "taco: A Tool to Generate Tensor Algebra Kernels", *Proc. 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*, Urbana-Champaign, USA, Oct. 30-Nov. 3, 2017, p. 943-948 (2017).

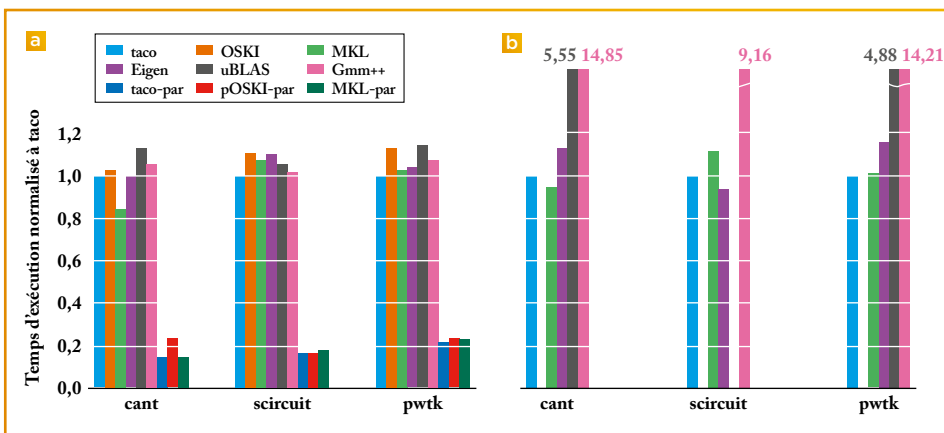


Figure 3

➤ (a) Temps d'exécution normalisés par rapport à *taco* d'un SpMV de forme $b_i = A_{ij} \times x_j$, obtenus avec des bibliothèques d'algèbre linéaire pour un ensemble de matrices; -par indique des temps d'exécution parallèle. Les temps d'exécution du code généré automatiquement par *taco* sur un ensemble de matrices (cant, scircuit, pwtk) sont comparables à ceux obtenus avec les codes optimisés manuellement des meilleures bibliothèques (MKL, OSKI, Eigen). (b) Comparaison de performances pour l'expression $A_{ij} = B_{ij} + C_{ij} + D_{ij}$, démontrant l'efficacité de *taco* sur des expressions tensorielles plus complexes.